



18BIG

Big Data Tools & Architecture

Lecture 01 — Foundations & Environment

www.bighub.ai

Your guides to applied AI



■ Your lecturers

Jan Kučera

- Senior Solution Architect · Data Engineering
- `jan.kucera@bighub.cz`

Petr Pokorný

- Senior Data Engineer
- `petr.pokorny@bighub.cz`

18BIG is taught together with **BigHub** — we build data & AI applications in energy, logistics, finance and medicine. This course is the hands-on, practical path into that world.

■ The course in one slide

Format

- Winter semester · **1 + 1** · pass/fail
- **12 sessions**, 1.5 h each
- Lecture **and** hands-on in the same block

What you build

- One **local lakehouse**, session by session
- Project: **E-shop Lakehouse**

How you pass

- **Actively complete all exercises**
- **Attendance 10+** (email us if you can't make it)
- More absences → **one extra standalone exercise**

Tools

- Everything **local**, `python` + `sql` first
- No cloud, no Kubernetes, one `uv sync`

■ What we'll cover (the arc)

Build the lakehouse — sessions 1–8

1. Foundations & environment · 2. Sources & Bronze · 3. Parquet & medallion · 4. Delta Lake
5. Silver: quality & contracts · 6. Core model & dbt · 7. Gold & viz · 8. Governance

Operate it — sessions 9–11

9. Orchestration (Dagster) · 10. Streaming & CDC · 11. MLOps

Look beyond + capstone — session 12

12. NoSQL/KV, ODS, distributed compute (Spark), project showcase

Every session **extends the same system** — you leave with one real pipeline, not 12 toy tasks.



Part 1

Why "Big Data"?

Your guides to applied AI



■ What makes data "big"?

The classic **V's** — when one machine and a spreadsheet stop being enough:

Volume

too big for one box

Velocity

arrives continuously

Variety

tables, JSON, events, files

Veracity

messy, needs trust

Value

only if we use it

"Without data, you're just another person with an opinion." — *W. E. Deming*

The goal of this course: turn raw, messy sources into **trustworthy data products**.

■ Why the classic Data Warehouse hits a wall

The DWH promise

- Clean, modeled, **schema-on-write**
- Great SQL & BI performance
- Strong governance

The pain

- **Expensive** storage & compute
- Rigid: schema *before* you load
- Struggles with JSON / events / files
- ML wants raw data, not just stars

So people built Data Lakes — cheap object storage, dump everything, decide the schema later.



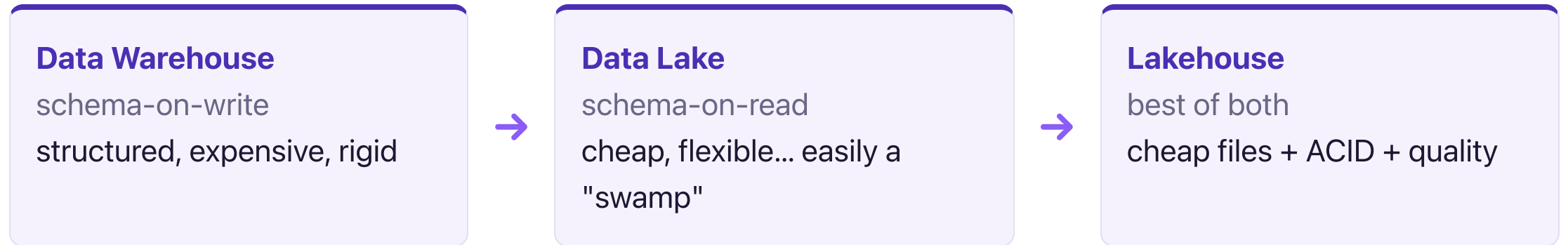
Evolution

DWH → Data Lake → Lakehouse

Your guides to applied AI



■ Three generations



	Warehouse	Lake	Lakehouse
Storage cost	high	low	low
Transactions (ACID)	yes	no	yes
Handles raw/semi-structured	weak	yes	yes
BI + ML on one copy	no	partial	yes

■ The Lakehouse idea

Keep data as **cheap open files** (Parquet), but add a **transaction layer** (Delta) that gives you ACID, updates, and time travel — plus SQL transformations, quality and governance on top.

You get

- One copy for BI *and* ML
- Open formats, no lock-in
- Warehouse reliability at lake cost

We'll build exactly this

- Files: **Parquet**
- Table layer: **Delta Lake**
- SQL & transforms: **DuckDB + dbt**

■ Where does this run? PaaS / IaaS / Local

- **IaaS** — you rent machines, run everything yourself (VMs, clusters).
- **PaaS** — managed platforms: *Databricks, Snowflake, Azure Synapse/Fabric*.
- **Local** — your laptop, no server at all. ← **this course**

We learn **locally** because the **concepts are identical**. What you learn here maps 1:1 onto Databricks / Snowflake / Fabric — minus the credentials, clusters and cost.



The project

E-shop Lakehouse

Your guides to applied AI



■ Business-Driven Development

We start from a **business question**, not a technology.

"Who are our customers, and how is our business doing?"

Everything we build this semester answers questions like that — revenue trends, top products, churn — by turning raw e-shop sources into clean, queryable **data products**.

Technology is the means, not the goal

■ The medallion architecture



- **Bronze** — *"what arrived, we don't touch."*
- **Silver** — cleaned, typed, deduplicated, **bad data quarantined**.
- **Gold** — aggregated marts consumed by BI, ML and apps.

Each layer = one (or two) sessions. Today we set the stage.

■ Our local stack

Area	Tool	Why
Environment	Python 3.12 + uv	fast, reproducible, one command
DataFrame	Polars	fast, simple, less magic
SQL / analytics	DuckDB	reads Parquet directly, no server
Storage	Parquet	columnar, compressed open files
Table format	Delta Lake	ACID, upsert, time travel — no JVM
Transforms	dbt-duckdb	SQL models, tests, lineage
Quality	Pandera	data contracts in Python

Later: Dagster (orchestration) · MLflow/Evidently (MLOps) · Streamlit (viz).

■ Why this backbone

DuckDB + Polars + Parquet + Delta + dbt — cheap, local, Python-friendly, yet enough to reach real lakehouse, data-engineering and MLOps concepts.

```
-- No PostgreSQL server. No Spark cluster. No cloud. Just a file:  
SELECT customer_id, SUM(amount) AS revenue  
FROM 'data/lake/silver/orders/*.parquet'  
GROUP BY customer_id;
```

You focus on **ideas**, not on operating infrastructure.



Hands-on

Set up & first queries

Your guides to applied AI



■ Environment in three commands

```
uv sync                # install the course backbone (one shot)
uv run eshop datagen    # generate synthetic e-shop data → data/raw/
uv run eshop doctor     # verify everything works
```

What datagen creates

- customers.csv, products.csv, payments.csv
- shop.sqlite → orders, order_items

Deterministic

- Fixed seed=18 → same data for everyone
- Intentionally a bit **dirty** (we'll clean it in session 5)

■ Today's exercise

`sessions/01-foundations/exercise/explore.py` — fill in the `# TODO` s:

1. Count customers & products (SQL over CSV)
2. Daily revenue (join `orders` + `order_items`)
3. Top 5 products by revenue
4. Orders per status
5. Average order value (a CTE)
6. Save daily revenue to Parquet
7. **(harder)** Month-over-month growth with a window function `lag()`

```
uv run python sessions/01-foundations/exercise/explore.py
```

- **DuckDB reads anything — no server**

```
import duckdb

# CSV directly:
duckdb.sql("SELECT count(*) FROM read_csv_auto('data/raw/customers.csv')")

# SQLite table directly:
duckdb.sql("SELECT * FROM sqlite_scan('data/raw/shop.sqlite', 'orders')")

# ... and write results straight to Parquet:
duckdb.sql("SELECT 1 AS x").write_parquet("out.parquet")
```

This is the whole superpower of the local stack: **query files where they lie.**

■ Key takeaways

- Big Data = when one machine/spreadsheet stops being enough (**the V's**).
- **Lakehouse** = cheap open files + a transaction layer → warehouse reliability at lake cost.
- Concepts learned **locally** transfer directly to Databricks / Snowflake / Fabric.
- We build **one** system across 12 sessions: **Bronze → Silver → Gold**.
- Start from the **business question**; the tools serve it.

■ Next time

Session 02 — Data sources & Bronze ingestion

- Files, JDBC/SQLite, queues (preview)
- Integration patterns: full vs. incremental, batch vs. stream, **idempotence**
- Land raw data into **Bronze** with lineage metadata

Come with your environment working: `uv run eshop doctor` should print **✓ environment OK.**



bighub

Thank you

Let's build.

www.bighub.ai

Your guides to applied AI

